

Programming and Problem Solving

Assignment 1 --- Due Sunday, October 04, 2026

Part I

Please read carefully: You must submit the answers to all the questions below. However, this part will not be marked. Nonetheless, failing to submit this part fully will result in you missing 50% of the total mark of the assignment.

Question 1

- a) Given a list of integers of any size n ($n \geq 1$), develop an algorithm as **pseudocode** (not a program) that finds the **smallest and largest elements** in the list, using **only one pass** through the list and no additional data structures. For example, given the list [8, 3, 12, 5, 1, 9] the algorithm should output smallest 1 and largest 12. Your solution must work for lists containing positive numbers, negative numbers, and duplicate values.
- b) What is the time-complexity of your algorithm, in terms of Big-O?
- c) What is the space-complexity of your algorithm, in terms of Big-O?

Question 2

Given a string of random length containing uppercase letters, lowercase letters and digits, write an algorithm in **pseudo code** that rearranges the string according to the following rules: (1) All uppercase letters must appear first. (2) All lowercase letters must appear second. (3) All digits must appear last. (4) The characters within each group must be sorted in ascending order. If, given "b3A1cB2a", the algorithm should return ABabc123.

- a) What is the time complexity of your algorithm, in terms of Big-O?
- b) What is the space complexity of your algorithm, in terms of Big-O?

Question 3

Given an array of integers, design a **well-documented pseudocode algorithm** that finds the **first pair of consecutive elements whose sum is divisible by 5**. Once such a pair is found, the algorithm must display: the two values, their indices, their sum, and the result of dividing the sum by 5. The algorithm must **stop searching immediately after finding the first qualifying pair**. For example, given: [7, 4, 6, 8, 12, 3] the algorithm would identify 4 and 6, because $4 + 6 = 10$ and 10 is divisible by 5. The output could be Elements 4 and 6 at indices 1 and 2 have a sum of 10, which is divisible by 5. Result: 2

(Note: This is only an example. Your pseudocode must provide a general solution and must not refer specifically to these values.)

- i) Design the well-documented pseudocode algorithm described above. Your algorithm must also correctly handle the situation where no qualifying pair exists.
- ii) Briefly explain how your algorithm efficiently determines whether the sum of two consecutive elements is divisible by 5.
- iii) What is the time complexity of your algorithm in the worst case? Specify the complexity using Big-O notation and clearly explain how you obtained it.
- iv) What is the maximum stack growth of your algorithm? Explain clearly.

Part II

Purpose: Practically, this part of the assignment can be considered as *Programming Assignment # 0*! The purpose of this assignment is to help you review some of the main topics covered in previous courses, including classes, loops, arrays, arrays of objects, static attributes, and static methods.

General Guidelines When Writing Programs:

- Include the following comments at the top of your source codes

```
// .....  
// Assignment (include number)  
// © Your Name  
// Written by: (include your name and student id)  
// .....
```
- In a comment, give a general explanation of what your program does. As the programming questions get more complex, the explanations will get lengthier.
- Include comments in your program describing the main steps in your program.
- Display clear prompts for users when you are expecting the user to enter data from the keyboard.
- All output should be displayed with clear messages and in an easy-to-read format.
- End your program with a closing message so that the user knows that the program has terminated.

Problem Description

Wildfires are a major environmental concern in many regions of the world. Environmental monitoring organizations need systems that allow them to record wildfire incidents, monitor their progression, and identify incidents that may require additional attention.

You are hired by a **global environmental monitoring organization** to develop a Java console application that helps analysts manage and monitor wildfire incidents.

Your program will maintain information about different Wildfires, including their location, area burned, containment percentage, risk level, and whether an evacuation is required.

Part II A

Design and implement a class called **Wildfire** according to the following requirements:

- ✓ A Wildfire object has eight attributes, namely, a fireID (long), fireName (String), country (String), region (String), areaBurned (double), containmentPercentage (double), riskLevel (String), and evacuationRequired (boolean).
- Upon creation of a Wildfire object, the object must immediately be initialized with valid values for all its attributes. The design should provide different ways of creating a Wildfire object, including creating one with appropriate default values. (Hint: use constructors.)
- The design must allow enough flexibility so that the value of any of these attributes can be modified later except for fireID. For example, the containment percentage of a Wildfire may increase as firefighters make progress, or its risk level may change as conditions evolve. The design must also allow a user to obtain the value of any attribute. (Hint: use accessors and mutators.)

- The design must allow all attributes of a Wildfire object to be displayed at once using the `System.out.print()` or `System.out.println()`. (Hint: use the `toString()` method.)
- It is required to know how many Wildfire objects have been created at any point during the execution of the program. For this purpose, the Wildfire class must have a method called `getNumberOfWildfires()`. This method must return the number of Wildfire objects that have been created prior to the time the method is invoked. The method returns 0 if no Wildfire objects have been created when it is called. (Hint: use a static attribute and/or static method. You may add other attributes to the class as required.)
- It is required to compare two Wildfire objects for equality. Two Wildfire objects are considered equal if they have the same fire name, country, and region. The comparison of these attributes should not be affected by differences in uppercase and lowercase letters. (Hint: use the `equals()` method.)

Part II B

Write a driver program that contains the `main()` method and performs the following operations. (Note: You may have the `main()` method in a separate driver file or in the same file as the Wildfire class.)

- Display a welcome message.
- Prompt a user for maximum number of Wildfires (`maxFires`) that the system can manage. Create an array named, `WildfireDatabase`, with a capacity of `maxFires` to store all created Wildfire objects.
- Display a main menu (figure 1) with the following choices and keep prompting the user until they enter a number between 1 and 5 inclusive:

What do you want to do?

1. Register new Wildfires (password required).
2. Update an existing Wildfire (password required).
3. Display Wildfires by country.
4. Display Wildfires by containment percentage.
5. Exit.

Please enter your choice >

Figure 1. Main menu

- When option 1 is entered:
 - Prompt the user for a password. Create a constant variable containing the password "fire2026". Do not use another password, as using the same password will make grading and testing easier.
 - The user has a maximum of 3 attempts to enter the correct password. After the third incorrect attempt, the main menu in Figure 1 must be redisplayed.
 - Keep track of consecutive failed authentication attempts for option 1. If this process is repeated 3 times (i.e., after 9 consecutive incorrect password attempts), the program must display the following message:
"WARNING: Multiple unauthorized access attempts detected. Program terminating immediately!"

The program must then terminate.

- ✓ A successful password entry resets the consecutive failed-attempt counter.
- ✓ If the correct password is entered, ask the user how many Wildfire objects they would like to enter.
- ✓ Check whether there is enough space available in WildfireDatabase to add the requested number of objects.
- ✓ If ample space is available, allow user to enter the requested number of Wildfires.
- ✓ Otherwise, inform the user of the **maximum number of additional Wildfires that can currently be stored** in the array.
- ✓ For example: The Wildfire Database has space for only 3 additional Wildfires.
- ✓ For each new Wildfire, prompt the user to enter all the required information: Fire ID, Fire Name, Country, Region, Area Burned, Containment Percentage, Risk Level, Evacuation Required.
- ✓ Before adding a new Wildfire, verify that its fireID is not already being used by another Wildfire object in WildfireDatabase.
- ✓ If the entered fireID already exists, inform the user and continue prompting until a unique ID is entered.
- ✓ Make sure that:
 - areaBurned is not negative;
 - containmentPercentage is between **0.0 and 100.0 inclusive**;
 - riskLevel is one of "Low", "Moderate", "High", or "Extreme".

➤ When option 2 is entered:

- Prompt the user for the password. Use the same password ("fire2026"). Again, the user has a maximum of **3 attempts** to enter the correct password. After the third incorrect attempt, simply redisplay the main menu shown in Figure 1. Unlike option 1, incorrect password attempts for option 2 **do not contribute toward program termination**.
- Once the correct password is entered, prompt the user to enter the fireID of the Wildfire they wish to update. If there is no Wildfire object with the specified fireID, display a message asking whether the user wishes to 1. Enter another Fire ID or 2. Return to the main menu
- If a Wildfire with the specified fireID is found, display its current information in the following format:

Wildfire: #x (index of the wildfire object in the WildfireDatabase array)

Fire ID: fireID of the Wildfire

Fire Name: fireName of the Wildfire

Country: country where the Wildfire is located

Region: province/state/region where the Wildfire is located

Area Burned: areaBurned in hectares

Containment Percentage: containmentPercentage %

Risk Level: Low / Moderate / High / Extreme

Evacuation Required: true / false

- Then ask the user which attribute they wish to modify by displaying the following menu.

What information would you like to change?

1. Fire Name
2. Country
3. Region
4. Area Burned
5. Containment Percentage
6. Risk Level
7. Evacuation Status
8. Quit this operation

Please enter your choice (1-8)>

Figure 2. Update menu

- Each time the user is prompted for a choice, ensure that a number from **1 to 8 inclusive** is entered. Otherwise, continue prompting until a valid choice is entered.
 - Once a valid choice from **1 to 7** is entered, prompt the user for the new value and modify the corresponding attribute.
 - Apply the same validation rules used when creating a Wildfire object.
 - After an attribute has been modified, display **all attributes of the updated Wildfire object** to show that the modification was successful.
 - Continue displaying the update menu and allowing additional modifications until the user selects **choice 8**.
 - The fireID must **not be modifiable**.
- When option 3 (in the main menu shown in figure. 1) is entered,
- Prompt the user to enter a **country name**.
 - Search WildfireDatabase and display the information of **all available Wildfire objects located in the specified country**.
 - The country comparison should be **case-insensitive**. For example, "Canada", "CANADA", and "canada" should be treated as the same country.
 - If no Wildfire is found in the specified country, display an appropriate message.
- Hint:** Create a static method called findWildfiresByCountry() that accepts required parameters and performs the search.
- When option 4 (in the main menu shown in figure. 1) is entered,
- Prompt the user to enter a **maximum containment percentage**.
 - Ensure that the entered value is between **0.0 and 100.0 inclusive**.
 - Display all **Wildfire** objects whose **containmentPercentage is less than or equal to the entered value**.
 - For example, if the user enters: 30.0, a Wildfire that is 25.0% contained must be displayed, whereas a Wildfire that is 75.0% contained must not be displayed.
 - If no Wildfire objects satisfy the specified condition, show an appropriate message.

- When option 5 (in the main menu shown in figure. 1) is entered,
 - Display an appropriate closing message, such as: “Thank you for using the Global Wildfire Monitoring System.” Then terminate the program.

SUBMISSION INSTRUCTIONS

Submission format: All assignment-related submissions must be adequately archived in a ZIP file using your ID and last name as file name. The submission itself must also contain your name(s) and student ID. Use your “official” name only – no abbreviations or nick names; capitalize your “last” name. Inappropriate submissions will be heavily penalized.

IMPORTANT: For Part II of the assignment, a demo for about 10 to 15 minutes will take place with the marker. You **must** attend the demo and be able to explain your program to the marker. The schedule of the demos will be determined and announced by the markers, and students must reserve a time slot for the demo.

Now, please read very carefully:

- **If you fail to demo, a zero mark is assigned regardless of your submission.**
- **If you book a demo time, and do not show up, for whatever reason, you will be allowed to reschedule a second demo but a penalty of 50% will be applied.**
- **Failing to demo at the second appointment will result in zero marks and no more chances will be given under any conditions.**

EVALUATION CRITERIA

IMPORTANT: **Part I** must fully be submitted. Failure to submit that part will cost 50% of the total marks of the assignment!

Part II.A (Class AIModel)	4 pts
Default & other constructors	1 pt
Accessor method for static attribute	1 pt
equals, toString and static attributes/methods	2 pts
Part II.B (Driver & other static methods)	6 pts
Handling of password	1 pt
Handling of option 1	1.5 pts
Handling of option 2	1.5 pts
Handling of option 3	0.75 pt
Handling of option 4	0.75 pt
Handling of option 5	0.5 pt